

Below are **C programs for Simple Queue, Circular Queue, and Priority Queue**, written in a lab/exam-friendly **menu-driven** format.

1. Simple Queue Using Array

C Program

C

```
#include <stdio.h>

#define MAX 5

int queue[MAX];
int front = -1;
int rear = -1;

/* Insert an element */
void enqueue(int value)
{
    if (rear == MAX - 1)
    {
        printf("Queue Overflow!\n");
        return;
    }

    if (front == -1)
        front = 0;

    rear++;
    queue[rear] = value;

    printf("%d inserted into the queue.\n", value);
}

/* Delete an element */
void dequeue()
{
    int value;

    if (front == -1 || front > rear)
    {
        printf("Queue Underflow!\n");
        return;
    }

    value = queue[front];
```

```

        if (front == rear)
        {
            front = -1;
            rear = -1;
        }
        else
        {
            front++;
        }

        printf("%d deleted from the queue.\n", value);
    }

    /* Display queue */
    void display()
    {
        int i;

        if (front == -1)
        {
            printf("Queue is empty!\n");
            return;
        }

        printf("Queue: ");

        for (i = front; i <= rear; i++)
        {
            printf("%d ", queue[i]);
        }

        printf("\n");
    }

    /* View front element */
    void peek()
    {
        if (front == -1)
        {
            printf("Queue is empty!\n");
            return;
        }

        printf("Front element = %d\n", queue[front]);
    }

    int main()
    {
        int choice, value;

        while (1)
        {

```

```

printf("\n--- SIMPLE QUEUE ---\n");
printf("1. Enqueue\n");
printf("2. Dequeue\n");
printf("3. Peek\n");
printf("4. Display\n");
printf("5. Exit\n");

printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice)
{
    case 1:
        printf("Enter value: ");
        scanf("%d", &value);
        enqueue(value);
        break;

    case 2:
        dequeue();
        break;

    case 3:
        peek();
        break;

    case 4:
        display();
        break;

    case 5:
        return 0;

    default:
        printf("Invalid choice!\n");
}
}

return 0;

```

Complexity

- Enqueue: **O(1)**
- Dequeue: **O(1)**
- Peek: **O(1)**
- Display: **O(n)**

2. Circular Queue Using Array

The circular queue reuses the empty positions created after deletion.

C Program

C

```
#include <stdio.h>

#define MAX 5

int queue[MAX];
int front = -1;
int rear = -1;

/* Insert an element */
void enqueue(int value)
{
    /* Check for full queue */
    if ((rear + 1) % MAX == front)
    {
        printf("Circular Queue Overflow!\n");
        return;
    }

    /* First element */
    if (front == -1)
    {
        front = 0;
        rear = 0;
    }
    else
    {
        rear = (rear + 1) % MAX;
    }

    queue[rear] = value;

    printf("%d inserted into the circular queue.\n", value);
}

/* Delete an element */
void dequeue()
{
    int value;

    if (front == -1)
    {
        printf("Circular Queue Underflow!\n");
        return;
    }
}
```

```

    value = queue[front];

    /* Only one element */
    if (front == rear)
    {
        front = -1;
        rear = -1;
    }
    else
    {
        front = (front + 1) % MAX;
    }

    printf("%d deleted from the circular queue.\n", value);
}

/* Display queue */
void display()
{
    int i;

    if (front == -1)
    {
        printf("Circular Queue is empty!\n");
        return;
    }

    printf("Circular Queue: ");

    i = front;

    while (1)
    {
        printf("%d ", queue[i]);

        if (i == rear)
            break;

        i = (i + 1) % MAX;
    }

    printf("\n");
}

/* View front element */
void peek()
{
    if (front == -1)
    {
        printf("Circular Queue is empty!\n");
        return;
    }

```

```

        printf("Front element = %d\n", queue[front]);
    }

int main()
{
    int choice, value;

    while (1)
    {
        printf("\n--- CIRCULAR QUEUE ---\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                enqueue(value);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                peek();
                break;

            case 4:
                display();
                break;

            case 5:
                return 0;

            default:
                printf("Invalid choice!\n");
        }
    }

    return 0;
}

```

Important Formula

The circular movement is performed using:

C

```
rear = (rear + 1) % MAX;
```

and

C

```
front = (front + 1) % MAX;
```

Complexity

- Enqueue: **O(1)**
- Dequeue: **O(1)**
- Peek: **O(1)**
- Display: **O(n)**

3. Priority Queue Using Array

Here we use the convention:

Smaller priority number = higher priority

For example:

```
Priority 1 → Highest  
Priority 2  
Priority 3  
Priority 4 → Lowest
```

C Program

C

```
#include <stdio.h>

#define MAX 5

struct Item
{
    int data;
    int priority;
};
```

```

struct Item pq[MAX];
int size = 0;

/* Insert an element according to priority */
void enqueue(int value, int priority)
{
    int i;

    if (size == MAX)
    {
        printf("Priority Queue Overflow!\n");
        return;
    }

    /*
        Shift elements with lower priority
        to make space for the new element.
    */
    i = size - 1;

    while (i >= 0 && pq[i].priority > priority)
    {
        pq[i + 1] = pq[i];
        i--;
    }

    pq[i + 1].data = value;
    pq[i + 1].priority = priority;

    size++;

    printf("%d inserted with priority %d.\n", value, priority);
}

/* Delete highest-priority element */
void dequeue()
{
    int value, priority;
    int i;

    if (size == 0)
    {
        printf("Priority Queue Underflow!\n");
        return;
    }

    value = pq[0].data;
    priority = pq[0].priority;

    /* Shift remaining elements */
    for (i = 0; i < size - 1; i++)
    {

```



```

        pq[i] = pq[i + 1];
    }

    size--;

    printf("%d deleted (Priority = %d).\n", value, priority);
}

/* Display priority queue */
void display()
{
    int i;

    if (size == 0)
    {
        printf("Priority Queue is empty!\n");
        return;
    }

    printf("\nData\tPriority\n");
    printf("-----\n");

    for (i = 0; i < size; i++)
    {
        printf("%d\t%d\n", pq[i].data, pq[i].priority);
    }
}

/* View highest-priority element */
void peek()
{
    if (size == 0)
    {
        printf("Priority Queue is empty!\n");
        return;
    }

    printf("Highest Priority Element = %d\n", pq[0].data);
    printf("Priority = %d\n", pq[0].priority);
}

int main()
{
    int choice;
    int value, priority;

    while (1)
    {
        printf("\n--- PRIORITY QUEUE ---\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Peek\n");
    }
}

```

```

printf("4. Display\n");
printf("5. Exit\n");

printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice)
{
    case 1:
        printf("Enter value: ");
        scanf("%d", &value);

        printf("Enter priority (1 = highest): ");
        scanf("%d", &priority);

        enqueue(value, priority);
        break;

    case 2:
        dequeue();
        break;

    case 3:
        peek();
        break;

    case 4:
        display();
        break;

    case 5:
        return 0;

    default:
        printf("Invalid choice!\n");
}
}

return 0;
}

```

Example

Suppose we insert:

Data	Priority
10	3
20	1
30	2
40	1

The queue becomes:

20 (1) → 40 (1) → 30 (2) → 10 (3)

The deletion order is:

20 → 40 → 30 → 10

For equal priority (20 and 40 both have priority 1), the program maintains **FIFO order**.

Complexity

For the sorted-array implementation:

- Enqueue: **O(n)** worst case
- Dequeue: **O(n)** in this implementation because elements are shifted
- Peek: **O(1)**
- Display: **O(n)**

4. Quick Comparison of the Three Programs

Operation	Simple Queue	Circular Queue	Priority Queue
Enqueue	O(1)	O(1)	O(n)
Dequeue	O(1)	O(1)	O(n)
Peek	O(1)	O(1)	O(1)
Display	O(n)	O(n)	O(n)
Ordering	FIFO	FIFO	Priority
Main issue	False overflow	More complex logic	Priority maintenance

Lab sequence to practice

1. Simple Queue using Array
2. Circular Queue using Array
3. Priority Queue using Array
4. Add `isEmpty()` and `isFull()` functions
5. Implement all three using **linked lists** as an advanced exercise.